One misconception is that most outages happen because of application bugs. In reality, many large production incidents are caused by infrastructure or networking behaviour. Examples include DNS instability, connection exhaustion, Kubernetes networking issues, load balancer misconfiguration, retry amplification, network partitions, or unexpected cross-region latency.

I have personally seen situations where application teams spent hours debugging APIs while the actual issue came from backend connection saturation between services.

Distributed systems fail in very strange ways sometimes. That is why observability matters so much.

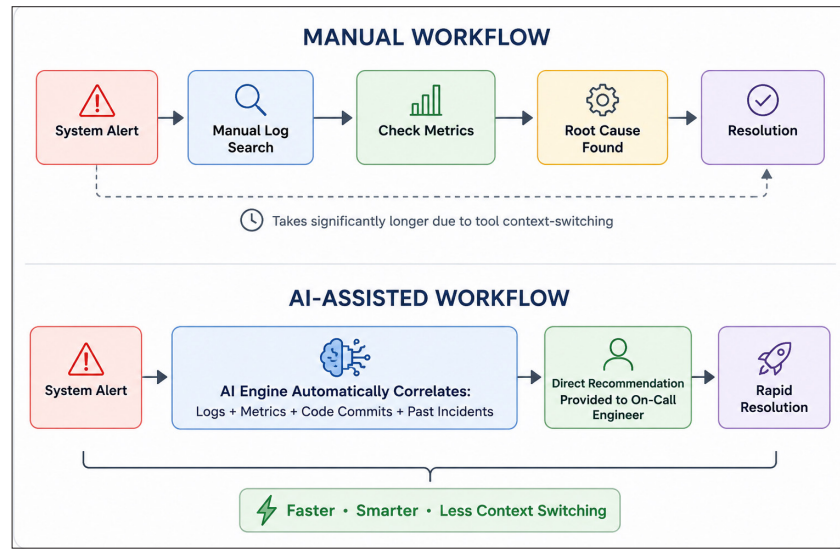


Figure 3: Manual workflow vs AI-assisted workflow

A simple example from production debugging

Consider a situation where API latency suddenly increases after a deployment. At first glance, dashboards may show elevated response times, increased CPU usage, and rising error rates.

Without observability, teams often start guessing. But tracing may reveal something different — a downstream service started retrying aggressively, database connections became exhausted, request queues started building, and eventually the upstream APIs slowed down.

The original deployment may not even belong to the affected API service.

This is exactly why distributed debugging is difficult. Failures propagate across systems very quickly. Figure 2 shows a typical distributed debugging workflow.

The role of AI in modern debugging

AI is now becoming part of observability and production operations. It doesn't replace engineers but definitely reduces investigation time.

Traditionally, an on-call engineer had to manually inspect dashboards, compare logs, correlate deployments, check traces, and review alerts one by one. Modern AI-assisted platforms now automate some of this correlation work. For example, AI systems can connect deployment timelines with error spikes, identify anomaly patterns, summarise incidents, reduce duplicate alerts, and suggest probable root causes.

This becomes valuable during high-pressure incidents where engineers are already overloaded with information.

Another major advantage is alert reduction. During large outages, one underlying issue may generate hundreds of alerts across different services. AI systems can group related failures into a single incident view, which reduces operational noise significantly.

However, AI has limitations. Most AI systems lack deep business understanding and system context. They may suggest technically correct fixes that create other operational or financial problems.

Human judgment still matters a lot.

Observability should be designed early

One common mistake is treating observability as something to be added later. In reality, observability should be part of the original architecture design. Good engineering teams usually think about metrics, tracing identifiers, health checks, structured logs, and alerting strategy while building the service itself. Retrofitting observability later becomes expensive and incomplete. The systems that recover fastest during incidents are usually the systems that were designed to be observable from day one.

Why human skills still matter

There is a lot of discussion now about AI replacing engineering work. I do not think distributed system debugging will disappear anytime soon.

Tools can recommend actions, but humans still need to understand architecture, evaluate risk, interpret unexpected behaviour, and make trade-off decisions. When completely new failures happen, engineers still rely on experience, intuition, and logical reasoning.

Large outages from providers like AWS, Azure, and banking infrastructure over recent years show that even highly automated systems still fail in unpredictable ways. During those situations, the most valuable engineers are usually the ones who stay calm, understand system interactions, read logs carefully, and think methodically under pressure.

Continue to page...98